James Ward
PHYC 236: Computational Physics
2/5/2026

1 Introduction

In this lab I show the importance of accounting for air resistance in computational modeling. This lab simulates a bicycle rider and their speeds throughout time while not accounting for drag compared to the results accounting for the force of drag. Air resistance or drag is used interchangeably throughout this paper, these terms are just referencing the effect that air has on the cyclist, this effect is a force that acts opposite to the direction the cyclist travels.

Methods

For this simulation I use the following equations:

$$F_{labeldrag} = \frac{1}{2} C_d \rho A v^2,$$

This equation calculates the force of drag which acts opposing the cyclist

$$F_{cyclist} = \min\left(F_{\max}, \frac{P}{v}\right) - \frac{1}{2} C_d \rho A v^2,$$

This equation calculates the bikers force when the biker is in motion.

$$F_{net} = F_{cyclist} - F_{drag,}$$

This equation calculates the net force by summing the force cyclist and the opposite acting drag force.

$$v_{i+1} = v_i + \frac{F_{net}}{m} \Delta t,$$

This equation calculates the speed while going through the loop it does this by adding the current speed to the product of the acceleration and timestep

For clarity I will explain the while loop, the while loop starts by calculating the drag force. Then if speed is greater than 0, then calculate the cyclist's speed based on their power and speed else the cyclist's speed is equal to their max force. This is like when siting on a bike and experiencing a lot of resistance when trying to take off from rest. Based off of the new cyclist's force and drag force the while loop calculates the net force and implements net force into an equation to solve for speed.

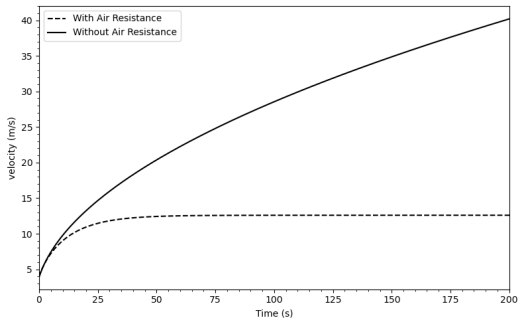

Figure 1: Change in Velocity of Cyclist Over time, with and without air resistance. The mass was 70 kg, drag coefficient was 0.5, the frontal area was 0.33 meters squared, time step was 0.1 m/s and the cyclist started at 4 m/s and air mass density = 1.2250. This resulted in a terminal velocity of 12.96 m/s

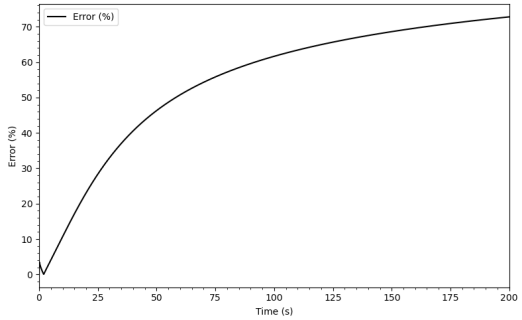

Figure 2: Percent error of theory compared to numerical results over time

Discussions

Figure 1 shows that the theoretical prediction is not accurate past the acceleration stage. The numerical results shows a terminal velocity of 12.96 m/s while the theoretical value gets to just under 40 m/s in the 200 second time frame. Figure 1 shows that the theoretical terminal velocity will continue to increase to infinity, while the numerical result that accounts for air resistance shows that there is a limit on the speed. From changing the timestep shows that it can affect the results for the terminal velocity. If I use a larger timestep the resulting terminal velocity will increase. Figure 2 shows that initially the error isn't terrible but continues to lose accuracy throughout the time of the cyclist journey.

Questions

Q1: My results are physical compared to the figure 2.2(From book) caption says about "13 m/s" while my results show 12.96 m/s.

Q2: Gaining 5 lbs will effect the acceleration and has very minimal effects on terminal velocity. Loses speed on terminal velocity by about 10e-4.

Q3: Using the same variables in Q1, I got that right around 400 watts is required if cycling alone to reach 13 m/s while cycling in a peloton only 270 watts is required to reach 13 m/s. The reason the cyclist's are in pelotons during races is because they are able to take advantage of this and expend less energy for the same amount of speed.

Appendix Code

```
# Lab #3 Bicycle Simulation
# James Ward 2/2/2026
# This program is the nuumerical and graphical results of a bicycle rider
    accounting for air resistance

# Import Libraries for Maatlab Plots and Math
import math as py
import matplotlib.pyplot as plt
import numpy as np
import matplotlib.style as style


# Set Variables, I added input option to make it easier to change values
#total_time = float(input("Enter the total simulation time in seconds: "))
    # Total simulation time in seconds
#time_step = 0.001 # Time step for simulation in seconds
#bicyclist_power = float(input("Enter the power output of the bicyclist in
    Watts: ")) # Power output of bicyclist in Watts
#bicyclist_mass = float(input("Enter the mass of the bicyclist in kg: "))
    # Mass of bicyclist in kg
#max_force = bicyclist_mass * 9.807 # Use weight as max applicable force.
    This is in (Newtons)

altitude = 0 # Altitude in meters
rho = 1.225 # constant for air mass density equation
h = 1600 # Scale height in meters
air_mass_density = rho * np.exp(-altitude/h) #calculate air mass density
    based on altitude using an exponential model
total_time = 200
time_step = .1 # Time step for simulation in seconds
bicyclist_power = 400 #power output of bicyclist in Watts
bicyclist_mass = 100# Mass of bicyclist and bike in kg
max_force = bicyclist_mass * 9.807 # Use weight as max applicable force.
    This is in (Newtons)


#drag related variables, I added input option to make it easier to change
    values
#cross_sectionArea = float(input("Enter the cross-sectional area of the
    bicyclist in m^2: ")) # unit m**-2; varies with bicyclist position, etc.
#air_mass_density = float(input("Enter the air mass density in kg/m^3: "))
    # unit kg/m**3; varies with temperature, altitude!
```

```
#drag_coefficient = float(input("Enter the drag coefficient of the
    bicyclist: ")) # really is unknown, depends on "aerodynamics"

# Drag related variables
cross_sectionArea = 0.4
drag_coefficient = 0.5 # really is unknown, depends on "aerodynamics"

# Calculations
denominator = drag_coefficient * air_mass_density * cross_sectionArea #
    Calculate the denominator for the terminal velocity calculation
terminal_velocity = (2.0 * bicyclist_power / denominator) ** (1.0 / 3.0) #
    Calculate terminal velocity
Fnet = (bicyclist_power/terminal_velocity) - (0.5 * drag_coefficient *
    air_mass_density * cross_sectionArea * terminal_velocity**2) #
    Calculate net force at terminal velocity

# initial condition:
initial_speed = 4 # in m/s

# set current number and time variables to initial values
speed = initial_speed
time = 0
no_Drag = initial_speed # add theoretical limit without drag as a check.

# BEGIN OUTPUT
print("Bicycle speed simulation")
print("Bicyclist power: ", bicyclist_power, "W")
print("Bicyclist mass: ", bicyclist_mass, "kg")
print("Bicyclist drag coefficient: ", drag_coefficient)
print("Bicyclist cross sectional area: ", cross_sectionArea, "m^2")
print("Mass density of air: ", air_mass_density, "kg/m^3")
print("Initial speed: ", initial_speed, "m/s")
print("Time step for simulation: ", time_step, "s")
print("Length of simulation: ", total_time, "s")


# Variables for MATLAB plot data
time_data = []
speed_data = []
theory_data = []
error_data = []

# Get Initial
time_data.append(time)
speed_data.append(speed)
```

```
theory_data.append(no_Drag)
error_data.append(0.0)

# EULER LOOP
while (time < total_time):
    # new values of velocity and time
   drag_force = 0.5 * drag_coefficient * air_mass_density *
       cross_sectionArea * speed * speed

    # limit force to a maximum of max_force
   if (speed > 0.0):
        bicyclist_force = min(max_force, bicyclist_power / speed) #
            Calculate the bicyclist force based on power and speed

   else:
        bicyclist_force = max_force # Zero drag at zero speed, so no drag
            force


   net_force = bicyclist_force - drag_force #Net total force subtract drag
       becuase it acts in oppisite direction
   speed = speed + (net_force / bicyclist_mass) * time_step # Updates speed
       by taking current speed and adding it with acceleration*time step

   theory = (
        (initial_speed * initial_speed) +
        (2.0 * bicyclist_power * time) / bicyclist_mass)**(0.5)

   time= time + time_step # Update time for loop

   time_data.append(time)
   speed_data.append(speed)
   theory_data.append(theory)
   error_data.append(abs((speed - theory) / theory) * 100.0)

print("Time =", float(time), "Speed =", speed, "m/s", "No drag =", theory,
    "m/s")


# Set up plots, Plots I used for figure 1, 2, and 3 they all used same
    code just different variables
plt.figure(figsize=(8, 5))
plt.plot(time_data, speed_data, label="With Air Resistance", color =
    'black', linestyle='--') # plots the numerical solution with -- lines
    style
```

```
plt.plot(time_data, theory_data, label="Without Air Resistance", color =
    'black', linestyle='-') # plots the theoretical solution with - line
    style

# I chose to not plot the error because it makes the graph hard to read
    but uncomment the line below is how I got the plot for figure 4
#plt.plot(time_data, error_data, label="Error (%)", color = 'black') #
    plots the numerical solution
plt.xlabel("Time (s)")
plt.ylabel("velocity (m/s)")
plt.xlim(0, total_time)
plt.minorticks_on()
plt.legend()
plt.tight_layout()
plt.show()
```